

实验 3 复杂查询和触发器

班级：07112304

学号：1120233329

姓名：陈墨霏

一、实验目的

掌握 SQL 嵌套查询和集合查询等各种高级查询的设计方法；掌握数据库触发器的设计和使用方法。

二、实验内容

1. 请用 SQL 语言实现以下查询

- (1) 查询订购了“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客的信息。
- (2) 查询没有购买过“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客的信息。
- (3) 查询订单平均金额超过 500 元的顾客中的中国籍顾客的信息。
- (4) 查询顾客“薛融”订购过而“宣荣揣”没有订购过的零件的信息。

请将查询结果截图附在实验报告中（可能会有查询结果为空，但也不一定）

2. 请建立以下触发器

- (1) 在 Lineitem 表上定义一个 INSERT 触发器，当增加一项订单明细时，自动修改订单表 Orders 中的订单总金额，以保持数据的一致性。（增加的那一部分金额为：零售价×数量×折扣）
- (2) 在 Lineitem 表上定义一个 BEFORE UPDATE 触发器，当修改订单明细中的数量时，先检查零件供应表 PartSupp 中的可用数量是否足够。

触发器建好之后，请进行实验以验证触发器建立是否正确，并将结果截图和必要的文字解释附在实验报告中。

三、实验步骤

3.1 SQL 语句实现复杂查询

发起查询请求前，注意先选择架构：

```
USE tpch;
```

3.1.1 查询请求一

查询订购了“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客的信息。
--

题目分析

可进行多次表连接操作，之后筛选所需顾客信息。具体步骤如下：

1. 使用 JOIN 子句连接 `customer`、`orders`、`lineitem`、`supplier` 和 `part` 表，查询出所有购买过“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客。
2. 在 WHERE 子句中添加条件，筛选出符合要求的记录。

编写查询语句

```
-- search1.sql

SELECT DISTINCT c.*
FROM customer c
JOIN orders o ON o.customer_id = c.customer_id
JOIN lineitem l ON l.order_id = o.order_id
JOIN supplier s ON s.supplier_id = l.supplier_id
JOIN part p ON p.part_id = l.part_id
WHERE s.supplier_name = '徐州市泰力公司矿山设备四厂'
      AND p.part_name = '活塞式减压阀';
```

3.1.2 查询请求二

查询没有购买过“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客的信息。

题目分析

先查询出所有购买过“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客，然后使用 NOT EXISTS 子句来筛选出没有购买过的顾客。具体步骤如下：

1. 使用 JOIN 子句连接 `customer`、`orders`、`lineitem`、`supplier` 和 `part` 表，查询出所有购买过“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客。
2. 使用 NOT EXISTS 子句，筛选出没有购买过的顾客。

编写查询语句

```
-- search2.sql

SELECT DISTINCT c.*
```

```
FROM customer c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    JOIN lineitem l ON l.order_id = o.order_id
    JOIN supplier s ON s.supplier_id = l.supplier_id
    JOIN part p ON p.part_id = l.part_id
    WHERE o.customer_id = c.customer_id
        AND s.supplier_name = '徐州市泰力公司矿山设备四厂'
        AND p.part_name = '活塞式减压阀'
);
```

3.1.3 查询请求三

查询订单平均金额超过 500 元的顾客中的中国籍顾客的信息。

题目分析

可以使用 JOIN 子句连接 `customer`、`orders` 和 `nation` 表，然后在 WHERE 子句中添加条件，筛选出符合要求的记录。具体步骤如下：

1. 使用 JOIN 子句连接 `customer`、`orders` 和 `nation` 表，查询出所有顾客的信息。
2. 在 WHERE 子句中添加条件，筛选出符合要求的记录。其中，需要使用 GROUP BY 以及 HAVING 子句，筛选出订单平均金额超过 500 元的顾客。

编写查询语句

```
-- search3.sql

SELECT c.*
FROM customer c
JOIN nation n ON c.nation_id = n.nation_id
WHERE n.nation_name = '中国'
    AND c.customer_id IN (
        SELECT o.customer_id
        FROM orders o
        GROUP BY o.customer_id
        HAVING AVG(o.total_price) > 500
    );
```

```
HAVING AVG(o.total_amount) > 500  
);
```

3.1.4 查询请求四

查询顾客“薛融”订购过而“宣荣揣”没有订购过的零件的信息。

题目分析

可以使用 JOIN 子句连接 `part`、`lineitem`、`orders` 和 `customer` 表，然后在 WHERE 子句中添加条件，筛选出符合要求的记录。具体步骤如下：

1. 使用 JOIN 子句连接 `part`、`lineitem`、`orders` 和 `customer` 表，结合 WHERE 子句，分别筛选出“薛融”和“宣荣揣”订购过的零件。
2. 使用 EXCEPT 子句计算差集，筛选出“薛融”订购过而“宣荣揣”没有订购过的零件。

编写查询语句

```
-- search4.sql  
  
SELECT p.*  
FROM part p  
JOIN lineitem l ON l.part_id = p.part_id  
JOIN orders o ON o.order_id = l.order_id  
JOIN customer c ON c.customer_id = o.customer_id  
WHERE c.name = '薛融'  
  
EXCEPT  
  
SELECT p.*  
FROM part p  
JOIN lineitem l ON l.part_id = p.part_id  
JOIN orders o ON o.order_id = l.order_id  
JOIN customer c ON c.customer_id = o.customer_id  
WHERE c.name = '宣荣揣'
```

3.2 触发器的建立

3.2.1 建立 INSERT 触发器 trig-update-orders-total-amount

在 Lineitem 表上定义一个 INSERT 触发器，当增加一项订单明细时，自动修改订单表 Orders 中的订单总金额，以保持数据的一致性。（增加的那一部分金额为：零售价×数量×折扣）

题目分析

触发器的作用是当在 lineitem 表中插入新记录时，自动更新 orders 表中相对应的 total_amount 字段。具体步骤如下：

1. 检查插入的 part_id 是否在 part 表中存在，如果不存在，则抛出异常。
2. 如果 part_id 存在，则获取该零件的零售价。
3. 检查 orders 表中是否存在对应的 order_id，如果不存在，则插入一条新的订单记录，初始 total_amount 为 0。
4. 更新 orders 表中对应的 total_amount，计算公式为：零售价 × 数量 × 折扣。

触发器创建 SQL 语句和说明

```
-- trig_update_orders_total_amount.sql

DELIMITER $$

CREATE TRIGGER trig_update_orders_total_amount
AFTER INSERT ON lineitem
FOR EACH ROW
BEGIN
    DECLARE v_price DECIMAL(10,2);
    DECLARE v_exists INT;

    -- 判断 part 是否存在
    SELECT COUNT(*) INTO v_exists
    FROM part
    WHERE part_id = NEW.part_id;

    -- 如果 part 不存在，则抛出异常
```

```
IF v_exists = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Invalid part_id: part does not exist';
END IF;

-- 获取零售价
SELECT retail_price INTO v_price
FROM part
WHERE part_id = NEW.part_id;

-- 判断 orders 是否存在
SELECT COUNT(*) INTO v_exists
FROM orders
WHERE order_id = NEW.order_id;

IF v_exists = 0 THEN
    -- 如果订单不存在，则插入新订单记录（初始 total_amount = 0）
    INSERT INTO orders(order_id, total_amount)
    VALUES (NEW.order_id, 0);
END IF;

-- 更新 total_amount
UPDATE orders
SET total_amount = total_amount + (v_price * NEW.quantity * NEW.discount)
WHERE order_id = NEW.order_id;
END$$

DELIMITER ;
```

几点说明（课堂上没有提到过的语法）：

1. **DECLARE**：用于声明变量，只能在 **BEGIN ... END** 块的最前面声明。。这里声明了两个变量 **v_price** 和 **v_exists**，分别用于存储零售价和判断记录是否存在的标志。
2. **SELECT COUNT(*) INTO v_exists**：用于查询记录的数量，并使用 **INTO** 将结果存储到变量 **v_exists** 中。

3. **DELIMITER \$\$**: 用于改变 SQL 语句的结束符, 以便在触发器中使用分号。在默认情况下, MySQL 使用分号 `;` 作为 SQL 语句的结束符号。但是——在创建存储过程、触发器、函数等复杂结构时, 代码中会包含多个分号, 这会导致 MySQL 误认为 SQL 语句结束了, 而引发错误。因此我们需要临时改变结束符号为 **\$\$**, 以便 MySQL 正确解析整个触发器的定义。在触发器定义结束后, 我们再将结束符号改回分号 `;`。
4. **SIGNAL SQLSTATE '45000'**: 用于抛出自定义异常。**45000** 是一个通用的 SQLSTATE 错误代码, 表示用户定义的异常。可以根据需要自定义错误消息。MySQL 不支持 **RAISE EXCEPTION** 语法, 因此使用 **SIGNAL** 来抛出异常。

3.2.2 建立 BEFORE UPDATE 触发器 trig-check-avail-qty

在 `Lineitem` 表上定义一个 BEFORE UPDATE 触发器, 当修改订单明细中的数量时, 先检查零件供应表 `PartSupp` 中的可用数量是否足够。

题目分析

触发器的作用是当在 `lineitem` 表中更新数量时, 检查 `partsupp` 表中对应的可用数量是否足够。具体步骤如下:

1. 检查更新的 `quantity` 是否发生变化, 如果没有变化, 则不进行检查。
2. 如果发生变化, 则查询 `partsupp` 表中对应的可用数量。
3. 如果可用数量小于更新后的数量, 则抛出异常, 提示库存不足。

关于可用数量的查找部分, 由于 `supplier_id` 和 `part_id` 总是同时出现在主键当中, 所以应该不存在多个供应商生产同一种零件的情况, 也不用担心会出现多个结果。另外, 由于在导入数据前已经设置好了外键约束, 所以也不用担心可用数量 `avail_qty` 查不到的问题。因此在下面的 PL/SQL 中, 可以直接使用 **SELECT ... INTO** 语句来获取可用数量, 而不进行其他检查和约束。

触发器创建 SQL 语句

```
-- trig_check_avail_qty.sql

DELIMITER $$

CREATE TRIGGER trig_check_avail_qty
BEFORE UPDATE ON lineitem
```

```
FOR EACH ROW
BEGIN
    -- 仅在 quantity 发生变更时检查
    DECLARE available_qty INT;

    IF NEW.quantity <> OLD.quantity THEN
        -- 查找可用库存数量
        SELECT ps.avail_qty
        INTO available_qty
        FROM partsupp ps
        WHERE ps.part_id = NEW.part_id
            AND ps.supplier_id = NEW.supplier_id;

        -- 判断库存是否足够
        IF available_qty < NEW.quantity THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Insufficient available quantity for the specified
part.';
        END IF;
    END IF;
END$$

DELIMITER ;
```


四、实验结果及分析

4.1 查询结果

4.1.1 查询结果一

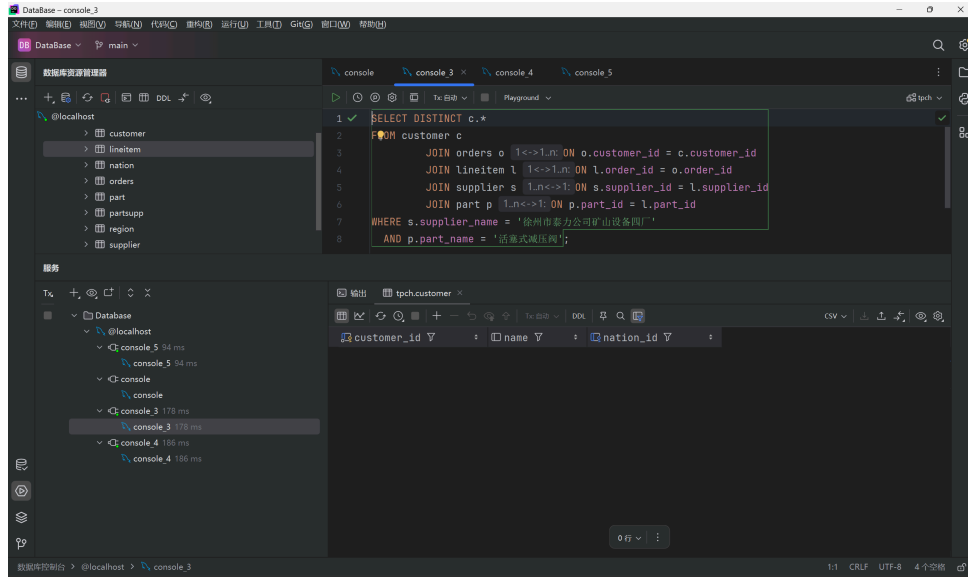


Figure 1: 查询订购了“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客的信息

结果分析

查询结果为空，说明没有顾客订购过“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”。

4.1.2 查询结果二

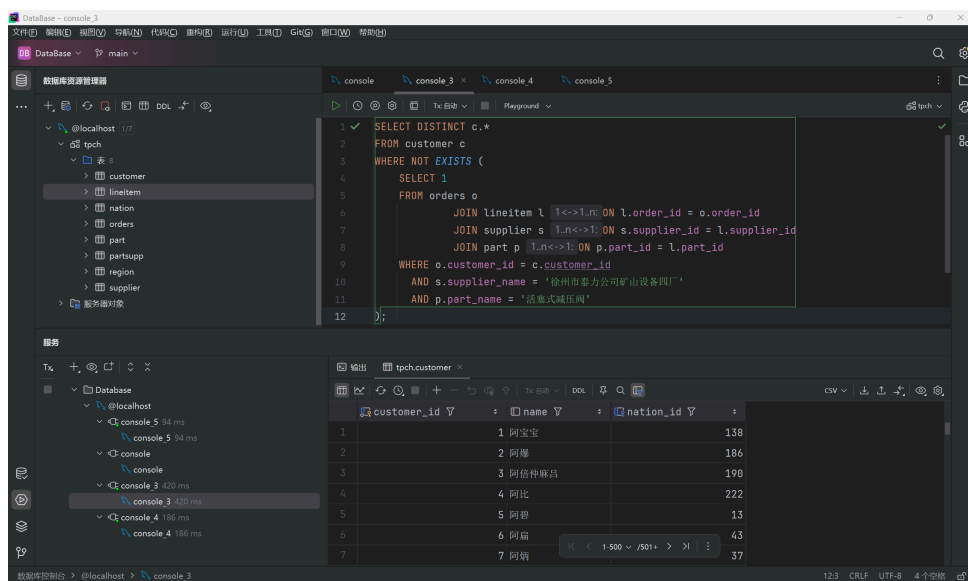


Figure 2: 查询没有购买过“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客的信息

结果分析

查询结果显示了“薛融”订购过而“宣荣揣”没有订购过的零件的信息。可以看到,虽然只有三条结果,但说明确实有一些零件是“薛融”订购过的,而“宣荣揣”没有订购过。

4.2 触发器建立与效果

4.2.1 触发器的建立

运行触发器创建 SQL 语句:

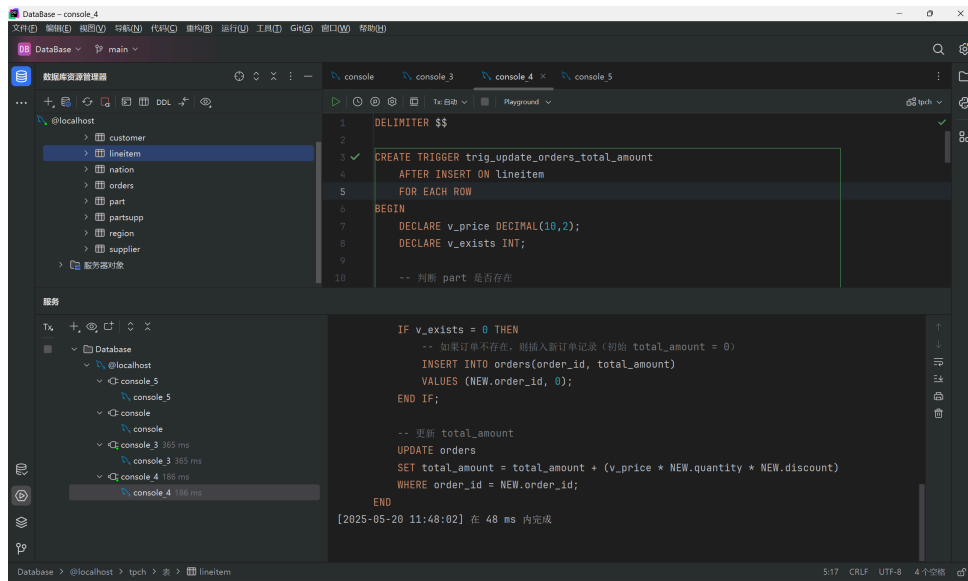


Figure 5: 触发器的建立

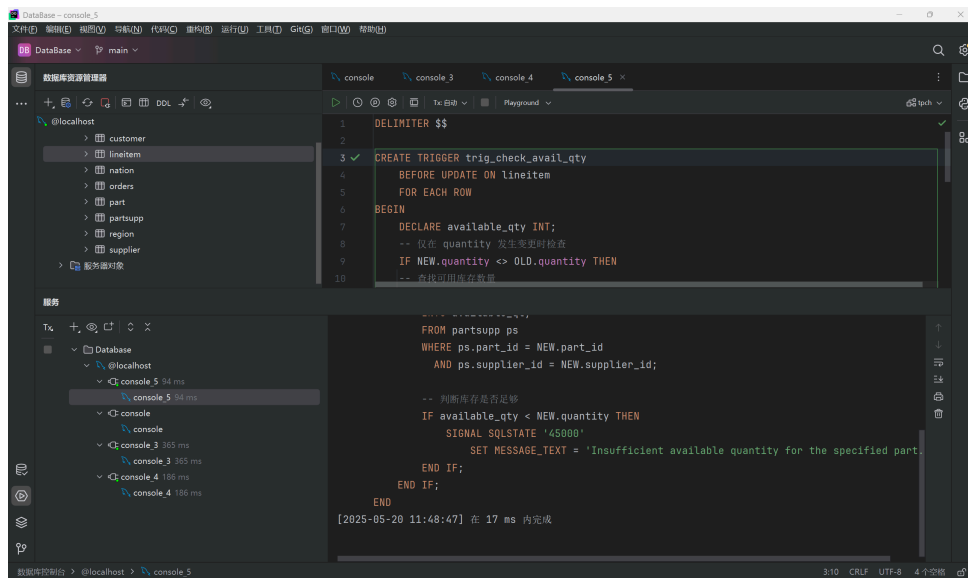


Figure 6: 触发器的建立

运行以下 SQL 语句查询 lineitem 上的触发器:

```
SHOW TRIGGERS WHERE `Table` = 'lineitem';
```

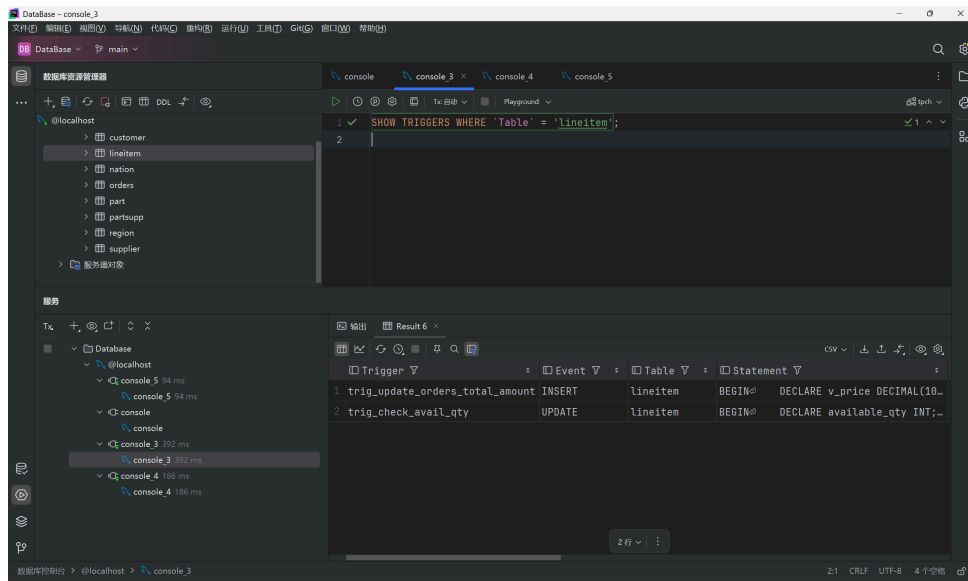


Figure 7: lineitem 上的触发器

4.2.2 触发器 trig-update-orders-total-amount 测试效果

以order_id为1的订单为例，首先查询该订单的基本信息：

```
SELECT * FROM orders WHERE order_id = 1;
```

得到的结果如下：

Table 1: 查询 order_id 为 1 的订单的基本信息

	order_id	customer_id	order_date	total_amount
1	1	320728	2014-05-14	437.00

可以看到，order_id为1的订单的总金额为437.00元。接下来，查询order_id为1的订单的详细信息：

```
SELECT * FROM lineitem WHERE order_id = 1;
```

得到的结果如下：

Table 2: 查询 order_id 为 1 的订单的详细信息

	order_id	part_id	supplier_id	quantity	return_flag	discount
1	1	42522	13503	2	0	0.27
2	1	44930	18848	1	0	0.09

可以看到，当前order_id为1的订单的详细信息有两条记录，分别是对于part_id为42522和44930的零件的订单明细。接下来，插入一条新的订单明细：

```
INSERT INTO lineitem (order_id, part_id, supplier_id, quantity, return_flag,
discount)
VALUES (1, 1, 1, 10, 0, 0.9);
```

接下来，再次查询`lineitem`表中`order_id`为1的订单的详细信息，得到的结果如下：

Table 3: 再次查询 `order_id` 为 1 的订单的详细信息

	order_id	part_id	supplier_id	quantity	return_flag	discount
1	1	42522	13503	2	0	0.27
2	1	44930	18848	1	0	0.09
3	1	1	1	10	0	0.9

可以看到，`lineitem`表中`order_id`为1的订单的详细信息多了一条记录，说明触发器已经成功执行。接下来，查询`orders`表中`order_id`为1的订单的基本信息，得到的结果如下：

Table 4: 再次查询 `order_id` 为 1 的订单的基本信息

	order_id	customer_id	order_date	total_amount
1	1	320728	2014-05-14	464.00

可以看到，`order_id`为1的订单的总金额已经更新为464.00元，说明触发器已经成功执行。接下来，查询`part`表中`part_id`为1的零件的零售价：

```
SELECT retail_price FROM part WHERE part_id = 1;
```

得到的结果如下：

Table 5: 查询 `part_id` 为 1 的零件的零售价

	retail_price
1	3.00

可以看到，`part_id`为1的零件的零售价为3.00元。代入公式计算发现，查询结果与之相符，证明触发器工作正常。

4.2.3 触发器 `trig-check-avail-qty` 测试效果

以`part_id`为1的订单明细为例，首先查询该订单明细的详细信息：

```
SELECT * FROM lineitem WHERE order_id = 1;
```

得到的结果如下：

Table 6: 查询 order_id 为 1 的订单的详细信息

	order_id	part_id	supplier_id	quantity	return_flag	discount
1	1	1	1	200	0	0.09

可以看到，当前order_id为1的订单的详细信息有一条记录，分别是对于part_id为1的零件的订单明细。接下来，查询partsupp表中part_id为1的零件的可用数量：

```
SELECT avail_qty FROM partsupp WHERE part_id = 1;
```

得到的结果如下：

Table 7: 查询 part_id 为 1 的零件的可用数量

	avail_qty
1	13

可以看到，part_id为1的零件的可用数量为13。接下来，尝试将quantity更新为20：

```
UPDATE lineitem
SET quantity = 20
WHERE order_id = 1 AND part_id = 1;
```

运行后，得到的结果如下：

```
[45000][1644] Insufficient available quantity for the specified part.
```

可以看到，更新失败，提示可用数量不足。接下来，将quantity更新为5：

```
UPDATE lineitem
SET quantity = 5
WHERE order_id = 1 AND part_id = 1;
```

运行后，得到的结果如下：

```
[2025-05-20 19:51:40] 7 ms 中有 1 行受到影响
```

可以看到，更新成功。接下来，再次查询lineitem表中part_id为1的订单的详细信息，得到的结果如下：

Table 8: 再次查询 part_id 为 1 的订单的详细信息

	order_id	part_id	supplier_id	quantity	return_flag	discount
1	1	1	5	0	0.09	

可以看到，`lineitem`表中`part_id`为 1 的订单的详细信息已经更新为 5，说明触发器已经成功执行。注意，如果此时查询`orders`表中`order_id`为 1 的订单的基本信息，会发现总金额没有更新，这是因为该触发器仅仅是检查可用数量是否足够，并不会自动更新`orders`表中的总金额。

五、实验收获与体会

通过本次实验，我深入学习了 SQL 的复杂查询语法，包括多表连接、嵌套查询、集合操作（如 EXCEPT）、分组与聚合等高级用法，提升了对数据库查询优化的理解。同时，首次实践了数据库触发器的编写与调试，体会到触发器在保证数据一致性和自动化处理中的重要作用。实验过程中遇到了一些语法和逻辑上的问题，通过查阅资料和反复调试，增强了独立分析和解决问题的能力。整体而言，本次实验不仅巩固了理论知识，也提升了实际操作能力，为后续数据库开发和管理打下了坚实基础。

附录：程序清单及说明

以下是本实验中涉及的主要程序文件及其说明：

1. **search/**：包含了所有的查询请求和对应的 SQL 语句。
 - (1) **search1.sql**：查询订购了“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客的信息。
 - (2) **search2.sql**：查询没有购买过“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客的信息。
 - (3) **search3.sql**：查询订单平均金额超过 500 元的顾客中的中国籍顾客的信息。
 - (4) **search4.sql**：查询顾客“薛融”订购过而“宣荣揣”没有订购过的零件的信息。
2. **trigger/**：包含了所有的触发器创建语句。
 - (1) **trig_update_orders_total_amount.sql**：在 Lineitem 表上定义一个 INSERT 触发器，当增加一项订单明细时，自动修改订单表 Orders 中的订单总金额，以保持数据的一致性。
 - (2) **trig_check_avail_qty.sql**：在 Lineitem 表上定义一个 BEFORE UPDATE 触发器，当修改订单明细中的数量时，先检查零件供应表 PartSupp 中的可用数量是否足够。

以上文件均已在实验中详细说明，并附有必要的注释以便理解和复现实验过程。